Rockchip GMS测试 多媒体模块帮助文档

文件标识：RK-PC-YF-E06

发布版本：V1.3.0

日期：2025-12-30

文件密级：□绝密 □秘密 □内部资料 ■公开

免责声明

本文档按"现状"提供，瑞芯微电子股份有限公司（"本公司"，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

"Rockchip"、"瑞芯微"、"瑞芯"均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2025 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址： 福建省福州市铜盘路软件园A区18号

网址： www.rock-chips.com

客户服务电话： +86-4007-700-590

客户服务传真： +86-591-83951833

客户服务邮箱： fae@rock-chips.com

前言

概述

本文提供了GMS测试过程中多媒体模块常见fail项的处理方法。

产品版本

Android 13及以上版本

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	刘兴亮	2025-02-13	初始版本
V1.1.0	刘兴亮	2025-04-18	新增失败项及其解决方法
V1.2.0	刘兴亮	2025-08-23	新增失败项及其解决方法
V1.3.0	刘兴亮	2025-12-30	新增失败项及其解决方法

目录

Rockchip GMS测试 多媒体模块帮助文档

1. 概述
2. 命令行测试
   - 2.1 测试环境准备
     - 2.1.1 资源包下载及push到机器
     - 2.1.2 测试应用安装及权限设置
   - 2.2 测试命令
     - 2.2.1 命令行说明
       - 2.2.1.1 命令行格式
       - 2.2.1.2 参数说明
       - 2.2.1.3 获取test_package/runner_class
       - 2.2.1.4 testcase
3. CTS常见fail项处理办法
   - 3.1 CtsMediaPlayerTestCases
     - 3.1.1 网络相关
   - 3.2 CtsMediaCodecTestCases
     - 3.2.1 网络相关
     - 3.2.2 色差
     - 3.2.3 禁用格式导致
   - 3.3 CtsVideoTestCases
     - 3.3.1 performance-point问题
   - 3.4 CtsMediaDecoderTestCases
     - 3.4.1 帧率问题
     - 3.4.2 色差问题
   - 3.5 CtsMediaAudioTestCases
     - 3.5.1 testTimestamp
     - 3.5.2 usb相关
     - 3.5.3 hdmi相关
   - 3.6 CtsMediaV2TestCases
   - 3.7 MctsMediaBetterTogetherTestCases[instant]
   - 3.8 MctsMediaV2TestCases
   - 3.9 CtsVideoCodecTestCases
4. GTS相关
   - 4.1 GTS本地媒体包测试
   - 4.2 常见fail项处理
     - 4.2.1 GtsExoPlayerTestCases
     - 4.2.2 WvtsDeviceTestCases

1. 概述

GMS测试包中包含大量多媒体相关（media/audio）的模块，测试过程中会出现各种各样的fail，一些fail项需要找代码开发者去解决，一些fail项可以通过修改配置自行解决。本文提供了测试模块快速测试方法以及常见fail项解决办法。

2. 命令行测试

GMS测试正常流程是需要机器连接VPN并通过GMS测试包完成测试，测试时间相对比较久，在debug过程中如果通过正常流程去验证修改，效率偏低。

可以通过adb shell用命令行的方式通过am instrument启动Instrumentation测试。

以下以CtsMediaV2TestCases为例详细说明。

2.1 测试环境准备

2.1.1 资源包下载及push到机器

CtsMediaV2TestCases对应资源包可以通过CtsMediaV2TestCases.dynamic查看

文件路径：

```
android-cts-15-r2/android-cts/testcases/CtsMediaV2TestCases
```

具体内容如下：

```
<dynamicConfig>
    <entry key="media_files_url">
      <value>https://dl.google.com/android/xts/cts/tests/media/CtsMediaV2TestCases-
5.0.zip</value>
    </entry>
</dynamicConfig>
```

下载CtsMediaV2TestCases-5.0.zip后解压，执行里面的copy_media.sh，会将资源文件push到测试机器对应目录下。

2.1.2 测试应用安装及权限设置

apk路径：

```
android-cts-15-r2/android-cts/testcases/CtsMediaV2TestCases/arm64/CtsMediaV2TestCases.apk
```

apk安装：

```
adb install CtsMediaV2TestCases.apk
```

权限设置：

```
Settings--Apps--All apps--android.mediav2.cts--Permissions--PHOTOS and videos--Allow--
CONFIRM
```

2.2 测试命令

以CtsMediaV2TestCases中某一个case为例单测说明。

某一单项case：

```
android.mediav2.cts.EncoderColorAspectsTest#testColorAspectsEndToEnd[176_c2.rk.avc.encoder_v
ideo/avc_unknown_bt709_unknown_surfacergb888_0-bframes]
```

测试完整命令：

```
am instrument -w -r -e class
android.mediav2.cts.EncoderColorAspectsTest#testColorAspectsEndToEnd[176_c2.rk.avc.encoder_v
ideo/avc_unknown_bt709_unknown_surfacergb888_0-bframes]
android.mediav2.cts/androidx.test.runner.AndroidJUnitRunner
```

测试结果：

```
rk3588_u:/ # am instrument -w -r -e class
android.mediav2.cts.EncoderColorAspectsTest#testColorAspectsEndToEnd[176_c2.rk.avc.encoder_v
ideo/avc_unknown_bt709_unknown_surfacergb888_0-bframes]
android.mediav2.cts/androidx.test.runner.AndroidJUnitRunner
INSTRUMENTATION_STATUS: class=android.mediav2.cts.EncoderColorAspectsTest
INSTRUMENTATION_STATUS: current=1
INSTRUMENTATION_STATUS: id=AndroidJUnitRunner
INSTRUMENTATION_STATUS: numtests=1
INSTRUMENTATION_STATUS: stream=
android.mediav2.cts.EncoderColorAspectsTest:
INSTRUMENTATION_STATUS:
test=testColorAspectsEndToEnd[176_c2.rk.avc.encoder_video/avc_unknown_bt709_unknown_surfacer
gb888_0-bframes]
INSTRUMENTATION_STATUS_CODE: 1
INSTRUMENTATION_STATUS: class=android.mediav2.cts.EncoderColorAspectsTest
```

```
INSTRUMENTATION_STATUS: current=1
INSTRUMENTATION_STATUS: id=AndroidJUnitRunner
INSTRUMENTATION_STATUS: numtests=1
INSTRUMENTATION_STATUS: stream=.
INSTRUMENTATION_STATUS:
test=testColorAspectsEndToEnd[176_c2.rk.avc.encoder_video/avc_unknown_bt709_unknown_surfacer
gb888_0-bframes]
INSTRUMENTATION_STATUS_CODE: 0
INSTRUMENTATION_RESULT: stream=

Time: 0.315

OK (1 test)


INSTRUMENTATION_CODE: -1
```

2.2.1 命令行说明

2.2.1.1 命令行格式

```
am instrument -w -r -e class testcase test_package/runner_class
```

2.2.1.2 参数说明

```
-w:保持adb shell打开直至测试完成
-r:以原始形式输出测试结果
-e <NAME> <VALUE>: set argument <NAME> to <VALUE>.提供过滤器和参数
```

2.2.1.3 获取test_package/runner_class

通过pm list instrumentation找test_package/runner_class

```
rk3588_u:/ # pm list instrumentation
instrumentation:android.mediav2.cts/androidx.test.runner.AndroidJUnitRunner
(target=android.mediav2.cts)
```

CtsMediaV2TestCases的test_package/runner_class是：
android.mediav2.cts/androidx.test.runner.AndroidJUnitRunner

2.2.1.4 testcase

- 整个模块

  testcase：android.mediav2.cts

- 模块中的一个大类集合

  testcase：android.mediav2.cts.EncoderColorAspectsTest

- 模块中的一个大类集合中的某一个case

  testcase：
  android.mediav2.cts.EncoderColorAspectsTest#testColorAspectsEndToEnd[176_c2.rk.avc.encoder_video/avc_unknown_bt709_unknown_surfacergb888_0-bframes]

3. CTS常见fail项处理办法

3.1 CtsMediaPlayerTestCases

3.1.1 网络相关

```
android.media.player.cts.StreamingMediaPlayerTest#testHTTP_H264Base_AAC_Video1
android.media.player.cts.StreamingMediaPlayerTest#testHTTP_H264Base_AAC_Video2
android.media.player.cts.StreamingMediaPlayerTest#testHTTP_MPEG4SP_AAC_Video1
android.media.player.cts.StreamingMediaPlayerTest#testHTTP_MPEG4SP_AAC_Video2
android.media.player.cts.StreamingMediaPlayerTest#testHTTP_H263_AMR_Video1
android.media.player.cts.StreamingMediaPlayerTest#testHTTP_H263_AMR_Video2
```

- 报错

  ```
  org.junit.runners.model.TestTimedOutException: test timed out after 600000
  milliseconds
  ```

- 原因分析

  这个case是播放在线视频，对网络要求比较高，如果网络环境不好，会出现这种10分钟超时的报错。

- 解决办法

  更换vpn或者通过本地媒体包的方式去测试

```
下载 https://storage.googleapis.com/cts_media/CtsMediaTestCases.zip
下载 https://storage.googleapis.com/android_media/cts/tests/tests/media/playe
r/CtsMediaPlayerTestCases-1.0.zip

adb push CtsMediaTestCases /sdcard/
cd CtsMediaPlayerTestCases-1.0
./copy_media.sh

run cts --module-arg CtsMediaPlayerTestCases:config-
url:https://storage.googleapis.com/cts_media/DynamicConfig_local.json -m
CtsMediaPlayerTestCases
```

3.2 CtsMediaCodecTestCases

3.2.1 网络相关

```
android.media.codec.cts.MediaCodecCapabilitiesTest#testAvcHigh31
android.media.codec.cts.MediaCodecCapabilitiesTest#testAvcHigh40
android.media.codec.cts.MediaCodecCapabilitiesTest#testAvcBaseline12
android.media.codec.cts.MediaCodecCapabilitiesTest#testAvcBaseline3
```

解决办法：

更换vpn或者使用本地媒体包的方式测试：

```
下载 https://storage.googleapis.com/cts_media/CtsMediaTestCases.zip
下载
https://storage.googleapis.com/android_media/cts/tests/tests/media/codec/CtsMediaCodecTestCa
ses-1.0.zip

adb push CtsMediaTestCases /sdcard/
cd CtsMediaCodecTestCases-1.0
./copy_media.sh

run cts --module-arg CtsMediaCodecTestCases:config-
url:https://storage.googleapis.com/cts_media/DynamicConfig_local.json -m
CtsMediaCodecTestCases
```

3.2.2 色差

- 测试项

  ```
  android.media.codec.cts.EncodeVirtualDisplayTest#testEncodeVirtualDisplay[0_c2.rk.avc.
  encoder_AVCLevel31]
  ```

- 报错

```
java.lang.RuntimeException: No match for color r=21 g=23 b=36
```

- 解决办法

  确认hwc是否存在如下修改，如果存在，请回退

```
diff --git a/drmhwctwo.cpp b/drmhwctwo.cpp
index 9e30cf68..15d7f211 100755
--- a/drmhwctwo.cpp
+++ b/drmhwctwo.cpp
@@ -2054,9 +2054,11 @@ HWC2::Error
DrmHwcTwo::HwcDisplay::PresentVirtualDisplay(int32_t* retire_fence)
     *retire_fence = -1;
     // 若虚拟屏图层数为0，则不做任何处理
+   bool has_valid_layer = true;
     if (!layers_.size()) {
         HWC2_LOGI_IF("display %" PRIu64 " layer size is %zu", handle_,
layers_.size());
-        return HWC2::Error::None;
+    // return HWC2::Error::None;
+   has_valid_layer = true;
     }
     if (bUseWriteBack_ && resource_manager_->isWBMode() && !resource_manager_-
>IsDisableHwVirtualDisplay()) {
@@ -2832,10 +2834,12 @@ HWC2::Error
DrmHwcTwo::HwcDisplay::ValidateVirtualDisplay(uint32_t* num_types, u
     if (LogLevel(DBG_INFO)) {
         DumpDisplayLayersInfo();
     }
-
+
+    bool has_valid_layer = true;
     if (!layers_.size()) {
         HWC2_LOGI("display %" PRIu64 " layer size is %zu, %s,line=%d", handle_,
layers_.size(), __FUNCTION__, __LINE__);
-        return HWC2::Error::None;
+        //return HWC2::Error::None;
+       has_valid_layer = false;
     }
     // 强制设置系统刷新为30帧
@@ -2897,46 +2901,50 @@ HWC2::Error
DrmHwcTwo::HwcDisplay::ValidateVirtualDisplay(uint32_t* num_types, u
             // }
             // 逐层匹配图层确认是否支持从WriteBack获取图像
-            for (std::pair<const hwc2_layer_t, DrmHwcTwo::HwcLayer>& l : layers_) {
-                DrmHwcTwo::HwcLayer& layer = l.second;
-                uint64_t buffer_id = 0;
-                const std::shared_ptr<LayerInfoCache> layer_info_cache =
layer.GetBufferInfo();
-                if (layer_info_cache != NULL) {
-                    buffer_id = layer_info_cache->uBufferId_;
-                }
-                // 判断信息当前仅需要 BufferId 与 图层名字
```

```
-                MirrorDisplayInfo_t info;
-                info.buffer_id = buffer_id;
-                info.name = layer.name();
-                if (!resource_manager_->IsWBMirrorDisplay(handle_, layer.z_order(),
info)) {
-                    HWC2_LOGD_IF("WB: Current display=%" PRIu64
-                                 " is not wb-display %d mirror display. disable VDS.
frame_no=%d",
-                                 handle_, WBDisplayId, frame_no_);
-                    bUseWriteBack_ = false;
-                }
-            }
+           if(has_valid_layer){
+               for (std::pair<const hwc2_layer_t, DrmHwcTwo::HwcLayer>& l : layers_)
{
+                   DrmHwcTwo::HwcLayer& layer = l.second;
+                   uint64_t buffer_id = 0;
+                   const std::shared_ptr<LayerInfoCache> layer_info_cache =
layer.GetBufferInfo();
+                   if (layer_info_cache != NULL) {
+                       buffer_id = layer_info_cache->uBufferId_;
+                   }
+                   // 判断信息当前仅需要 BufferId 与 图层名字
+                   MirrorDisplayInfo_t info;
+                   info.buffer_id = buffer_id;
+                   info.name = layer.name();
+                   if (!resource_manager_->IsWBMirrorDisplay(handle_,
layer.z_order(), info)) {
+                       HWC2_LOGD_IF("WB: Current display=%" PRIu64
+                                    " is not wb-display %d mirror display. disable
VDS. frame_no=%d",
+                                    handle_, WBDisplayId, frame_no_);
+                       bUseWriteBack_ = false;
+                   }
+               }
+           }
         }
     } else {
         bUseWriteBack_ = false;
         HWC2_LOGD_IF("WB display %d is invalid, disable HW VDS.", WBDisplayId);
     }
-
-    for (std::pair<const hwc2_layer_t, DrmHwcTwo::HwcLayer>& l : layers_) {
-        DrmHwcTwo::HwcLayer& layer = l.second;
-        if (bUseWriteBack_) {
-            if (layer.sf_type() == HWC2::Composition::Sideband) {
-                layer.set_validated_type(HWC2::Composition::Sideband);
-            } else {
-                layer.set_validated_type(HWC2::Composition::Device);
-            }
-        } else {
-            layer.set_validated_type(HWC2::Composition::Client);
-        }
```

```
-        //num_types 应该为发生改变的图层，不仅仅是Client图层
-        if(layer.type_changed() && (layer.hwc3_sf_type() !=
HWC3_COMPOSITION_DISPLAY_DECORATION)){
-            ++*num_types;
-        }
-    }
+
+   if(has_valid_layer){
+       for (std::pair<const hwc2_layer_t, DrmHwcTwo::HwcLayer>& l : layers_) {
+           DrmHwcTwo::HwcLayer& layer = l.second;
+           if (bUseWriteBack_) {
+               if (layer.sf_type() == HWC2::Composition::Sideband) {
+                   layer.set_validated_type(HWC2::Composition::Sideband);
+               } else {
+                   layer.set_validated_type(HWC2::Composition::Device);
+               }
+           } else {
+               layer.set_validated_type(HWC2::Composition::Client);
+           }
+           //num_types 应该为发生改变的图层，不仅仅是Client图层
+           if(layer.type_changed() && (layer.hwc3_sf_type() !=
HWC3_COMPOSITION_DISPLAY_DECORATION)){
+               ++*num_types;
+           }
+       }
+   }
     *num_requests = 0;
```

3.2.3 禁用格式导致

- 测试项

```
android.media.codec.cts.MediaCodecTest#testFlushAfterFirstBuffer
```

- 报错

```
E C2SoftAacDec: HE-AAC not supported
E MediaCodec: Codec reported err 0x80000000/UNKNOWN_ERROR, actionCode 0, while in
state 5/STARTING
```

- 解决办法

  确认frmeworks/av是否存在如下修改，如果有，请回退

```
diff --git a/media/codec2/components/aac/C2SoftAacDec.cpp
b/media/codec2/components/aac/C2SoftAacDec.cpp
index c770d0c8bf..529c7e01b4 100644
--- a/media/codec2/components/aac/C2SoftAacDec.cpp
+++ b/media/codec2/components/aac/C2SoftAacDec.cpp
@@ -52,6 +52,9 @@
```

```
 #define PROP_DRC_OVERRIDE_ENC_LEVEL "aac_drc_enc_target_level"
 #define PROP_DRC_OVERRIDE_EFFECT     "ro.aac_drc_effect_type"
+// property that indicating whether HE-AAC is enabled
+#define PROP_ENABLE_HE_AAC           "codec2_enable_heaac"
+
 namespace android {
 constexpr char COMPONENT_NAME[] = "c2.android.aac.decoder";
@@ -259,6 +262,11 @@ public:
     int32_t getDrcAlbumMode() const { return mDrcAlbumMode->value; }
     u_int32_t getMaxChannelCount() const { return mMaxChannelCount->value; }
     int32_t getDrcOutputLoudness() const { return (mDrcOutputLoudness->value <= 0 ? -
mDrcOutputLoudness->value * 4. + 0.5 : -1); }
+    bool isHEAACProfile() const {
+        return (mProfileLevel->profile == PROFILE_AAC_HE ||
+                mProfileLevel->profile == PROFILE_AAC_HE_PS ||
+                mProfileLevel->profile == PROFILE_AAC_XHE);
+    }
 private:
     std::shared_ptr<C2StreamSampleRateInfo::output> mSampleRate;
@@ -338,6 +346,12 @@ void C2SoftAacDec::onRelease() {
 status_t C2SoftAacDec::initDecoder() {
     ALOGV("initDecoder()");
     status_t status = UNKNOWN_ERROR;
+
+    if (mIntf->isHEAACProfile() && !property_get_bool(PROP_ENABLE_HE_AAC, false)) {
+        ALOGE("HE-AAC not supported");
+        return status;
+    }
+
     mAACDecoder = aacDecoder_Open(TT_MP4_ADIF, /* num layers */ 1);
     if (mAACDecoder != nullptr) {
         mStreamInfo = aacDecoder_GetStreamInfo(mAACDecoder);
@@ -872,6 +886,19 @@ void C2SoftAacDec::process(
                     work->result = C2_CORRUPTED;
                     return;
                 }
+            } else {
+                /*
+                 * Determines if the stream is HE-AAC encoded
+                 * HE-AAC (AAC+) uses SBR technology where the output sample rate
+                 * is exactly twice the core AAC sample rate.
+                 */
+                if (mStreamInfo->sampleRate == (2 * mStreamInfo->aacSampleRate)
+                        && !property_get_bool(PROP_ENABLE_HE_AAC, false)) {
+                    ALOGE("HE-AAC not supported");
+                    mSignalledError = true;
+                    work->result = C2_CORRUPTED;
+                    return;
+                }
             }
```

3.3 CtsVideoTestCases

3.3.1 performance-point问题

- 测试项

```
android.video.cts.CodecDecoderPerformanceTest#testPerformanceOfHardwareVideoDecoders[112(c2.rk.vp9.decoder_1_2.5)]
```

- 报错

```
java.lang.AssertionError: Unable to achieve the expected rate. DecodeMime: video/x-vnd.on2.vp9, Decoder: c2.rk.vp9.decoder, resolution: 2160p, Key-priority: 1 :: act/exp fps: 41.586958329867755/57.0
```

- 分析

rk356x对应的vendor/rockchip/common/vpu/etc/media_codecs_c2_rk356x.xml文件中c2.rk.vp9.decoder的配置如下：

```
        <MediaCodec name="c2.rk.vp9.decoder" type="video/x-vnd.on2.vp9">
            <Limit name="size" min="320x182" max="4096x2160" />
            <Limit name="alignment" value="2x2" />
            <Limit name="block-size" value="16x16" />
            <Limit name="block-count" range="1-32768" /> <!-- max 4096x2160 -->
            <Limit name="blocks-per-second" range="1-1966080" />
            <Limit name="frame-rate" range="1-60" />
            <Limit name="bitrate" range="1-20000000" />
            <Feature name="adaptive-playback" />
            <Limit name="concurrent-instances" max="32" />
            <Limit name="performance-point-3840x2160" value="60" />
        </MediaCodec>
```

Android支持的standard point如下：

```
List<VideoCapabilities.PerformancePoint> standardPoints = Arrays.asList(
      VideoCapabilities.PerformancePoint.UHD_240,/*3840x2160@240*/
      VideoCapabilities.PerformancePoint.UHD_200,/*3840x2160@200*/
      VideoCapabilities.PerformancePoint.UHD_120,/*3840x2160@120*/
      VideoCapabilities.PerformancePoint.UHD_100,/*3840x2160@100*/
      VideoCapabilities.PerformancePoint.UHD_60, /*3840x2160@60*/
      VideoCapabilities.PerformancePoint.UHD_50, /*3840x2160@50*/
      VideoCapabilities.PerformancePoint.UHD_30, /*3840x2160@30*/
      VideoCapabilities.PerformancePoint.UHD_25, /*3840x2160@25*/
      VideoCapabilities.PerformancePoint.UHD_24, /*3840x2160@24*/
      VideoCapabilities.PerformancePoint.FHD_240,/*1920x1080@240*/
      VideoCapabilities.PerformancePoint.FHD_200,/*1920x1080@200*/
      VideoCapabilities.PerformancePoint.FHD_120,/*1920x1080@120*/
      VideoCapabilities.PerformancePoint.FHD_100,/*1920x1080@100*/
      VideoCapabilities.PerformancePoint.FHD_60, /*1920x1080@60*/
```

```
        VideoCapabilities.PerformancePoint.FHD_50, /*1920x1080@50*/
        VideoCapabilities.PerformancePoint.FHD_30, /*1920x1080@30*/
        VideoCapabilities.PerformancePoint.FHD_25, /*1920x1080@25*/
        VideoCapabilities.PerformancePoint.FHD_24, /*1920x1080@24*/
        VideoCapabilities.PerformancePoint.HD_240, /*1280x720@240*/
        VideoCapabilities.PerformancePoint.HD_200, /*1280x720@200*/
        VideoCapabilities.PerformancePoint.HD_120, /*1280x720@120*/
        VideoCapabilities.PerformancePoint.HD_100, /*1280x720@100*/
        VideoCapabilities.PerformancePoint.HD_60,  /*1280x720@60*/
        VideoCapabilities.PerformancePoint.HD_50,  /*1280x720@50*/
        VideoCapabilities.PerformancePoint.HD_30,  /*1280x720@30*/
        VideoCapabilities.PerformancePoint.HD_25,  /*1280x720@25*/
        VideoCapabilities.PerformancePoint.HD_24,  /*1280x720@24*/
        VideoCapabilities.PerformancePoint.SD_60,  /*720x480@60*/
        VideoCapabilities.PerformancePoint.SD_50,  /*720x576@50*/
        VideoCapabilities.PerformancePoint.SD_48,  /*720x480@48*/
        VideoCapabilities.PerformancePoint.SD_30,  /*720x480@30*/
        VideoCapabilities.PerformancePoint.SD_25,  /*720x576@25*/
        VideoCapabilities.PerformancePoint.SD_24); /*720x480@24*/
```

报错分析如下：

```
exp: 60 * 0.95 = 57  act: 41.5
测试实际能达到的fps = 41.5 / 0.95 = 43.1
43介于30和50之间，若想测试通过且符合谷歌标准，performance-point-3840x2160的value需改成30
```

- 解决办法

```
diff --git a/vpu/etc/media_codecs_c2_rk356x.xml b/vpu/etc/media_codecs_c2_rk356x.xml
index b3288c6..0fb8e75 100644
--- a/vpu/etc/media_codecs_c2_rk356x.xml
+++ b/vpu/etc/media_codecs_c2_rk356x.xml
@@ -50,7 +50,7 @@
             <Limit name="bitrate" range="1-20000000" />
             <Feature name="adaptive-playback" />
             <Limit name="concurrent-instances" max="32" />
-            <Limit name="performance-point-3840x2160" value="60" />
+            <Limit name="performance-point-3840x2160" value="30" />
         </MediaCodec>
         <MediaCodec name="c2.rk.vp8.decoder" type="video/x-vnd.on2.vp8">
             <Limit name="size" min="2x2" max="1920x1088" />
```

注意：以上是以rk356x为例说明，其它芯片平台修改方法一致，请修改对应的media_codecs_c2_rkxxx.xml

3.4 CtsMediaDecoderTestCases

3.4.1 帧率问题

- 测试项

```
android.media.decoder.cts.VideoDecoderPerfTest#testPerf[0(c2.rk.avc.decoder:qvga)]
```

- 报错

```
E TestRunner: failed: testPerf[0(c2.rk.avc.decoder:qvga)]
(android.media.decoder.cts.VideoDecoderPerfTest)
E TestRunner: ----- begin exception -----
E TestRunner: java.lang.AssertionError: Expected achievable frame rates for
c2.rk.avc.decoder video/avc 320x240: [500.0, 560.0].
E TestRunner: Measured frame rate: [135.4609736934789, 135.4713543799203].
E TestRunner:  expected null, but was:<Expected achievable frame rates for
c2.rk.avc.decoder video/avc 320x240: [500.0, 560.0].
E TestRunner: Measured frame rate: [135.4609736934789, 135.4713543799203].
E TestRunner: >
E TestRunner:   at org.junit.Assert.fail(Assert.java:89)
E TestRunner:   at org.junit.Assert.failNotNull(Assert.java:756)
E TestRunner:   at org.junit.Assert.assertNull(Assert.java:738)
```

- 解决办法

  报错信息会提示配置，以rk3568对应的xml为例，配置的是[500.0, 560.0]，实际测试才[135.4609736934789, 135.4713543799203]，说明原来配置的偏大，重新配置将实际测试的结果包含进去即可，可配置[130, 140]，如下补丁所示：

```
diff --git a/vpu/etc/media_codecs_performance_rk356x.xml
b/vpu/etc/media_codecs_performance_rk356x.xml
index e763b0f..d5dafac 100644
--- a/vpu/etc/media_codecs_performance_rk356x.xml
+++ b/vpu/etc/media_codecs_performance_rk356x.xml
@@ -88,7 +88,7 @@
         </MediaCodec>
         <!-- c2_decoder performance list -->
         <MediaCodec name="c2.rk.avc.decoder" type="video/avc" update="true">
-            <Limit name="measured-frame-rate-320x240" range="500-560" />
+            <Limit name="measured-frame-rate-320x240" range="130-140" />
             <Limit name="measured-frame-rate-720x480" range="480-520" />
             <Limit name="measured-frame-rate-1280x720" range="310-315" />
             <Limit name="measured-frame-rate-1920x1080" range="164-164" />
```

  xml路径vendor/rockchip/common/vpu/etc，其它芯片平台修改对应xml即可，所有这类问题修改方法都一致。

3.4.2 色差问题

- 测试项1

```
android.media.decoder.cts.DecodeAccuracyTest#testGLViewLargerHeightDecodeAccuracy[7(c2
.android.vp9.decoder_vp9_82x144)]
```

- 报错

```
junit.framework.AssertionFailedError: With the best matched border crop (0.0, 0.0),
greatest pixel difference is 174 at (479, 317) which is over the allowed difference
105
```

- 解决办法

```
diff --git a/libs/gui/GLConsumerUtils.cpp b/libs/gui/GLConsumerUtils.cpp
index 2b89d82c29..a1c69e7d6d 100644
--- a/libs/gui/GLConsumerUtils.cpp
+++ b/libs/gui/GLConsumerUtils.cpp
@@ -85,19 +85,13 @@ void GLConsumer::computeTransformMatrix(float outTransform[16],
float bufferWidt
                 case PIXEL_FORMAT_BGRA_8888:
                     // We know there's no subsampling of any channels, so we
                     // only need to shrink by a half a pixel.
-                    // rk-code: 此修改会引入HWC合成与GPU合成效果差异问题，具体表现为
-                    // GPU合成与HWC合成切换过程中，画面会抖动
-                    // shrinkAmount = 0.5;
-                    // rk-code
+                    shrinkAmount = 0.5;
                     break;
                 default:
                     // If we don't recognize the format, we must assume the
                     // worst case (that we care about), which is YUV420.
-                    // rk-code: 此修改会引入HWC合成与GPU合成效果差异问题，具体表现为
-                    // GPU合成与HWC合成切换过程中，画面会抖动
-                    // shrinkAmount = 1.0;
-                    // rk-code
+                    shrinkAmount = 1.0;
                     break;
             }
         }
diff --git a/libs/nativedisplay/surfacetexture/SurfaceTexture.cpp
b/libs/nativedisplay/surfacetexture/SurfaceTexture.cpp
index c8f93eddc4..d3d4cbafdf 100644
--- a/libs/nativedisplay/surfacetexture/SurfaceTexture.cpp
+++ b/libs/nativedisplay/surfacetexture/SurfaceTexture.cpp
@@ -313,19 +313,13 @@ void SurfaceTexture::computeTransformMatrix(float
outTransform[16], const sp<Gra
                 case PIXEL_FORMAT_BGRA_8888:
                     // We know there's no subsampling of any channels, so we
                     // only need to shrink by a half a pixel.
```

```
-                    // rk-code: 此修改会引入HWC合成与GPU合成效果差异问题，具体表现为
-                    // GPU合成与HWC合成切换过程中，画面会抖动
-                    // shrinkAmount = 0.5;
-                    // rk-code
+                    shrinkAmount = 0.5;
                     break;
                 default:
                     // If we don't recognize the format, we must assume the
                     // worst case (that we care about), which is YUV420.
-                    // rk-code: 此修改会引入HWC合成与GPU合成效果差异问题，具体表现为
-                    // GPU合成与HWC合成切换过程中，画面会抖动
-                    // shrinkAmount = 1.0;
-                    // rk-code
+                    shrinkAmount = 1.0;
                     break;
             }
         }
```

- 测试项2

```
android.media.decoder.cts.DecodeAccuracyTest#testGLViewLargerWidthDecodeAccuracy[50(c2.rk.avc.decoder_h264_1920x1440)]
```

- 报错

```
junit.framework.AssertionFailedError: With the best matched border crop (0.0, 0.0), greatest pixel difference is 244 at (0, 210) which is over the allowed difference 105
```

- 解决办法

  vendor/rockchip/hardware/interfaces/codec2

```
diff --git a/component/osal/C2RKChipCapDef.cpp b/component/osal/C2RKChipCapDef.cpp
index 86f8955..bd7529e 100644
--- a/component/osal/C2RKChipCapDef.cpp
+++ b/component/osal/C2RKChipCapDef.cpp
@@ -317,7 +317,7 @@ uint32_t C2RKChipCapDef::getFbcMinStride(uint32_t fbcMode) {
         if (fbcMode == C2_COMPRESS_RFBC_64x4) {
             minStride = 4096;
         } else {
-            minStride = 1920;
+            minStride = 2100;
         }
     }
```

  这个fail在rk356x比较容易出现

3.5 CtsMediaAudioTestCases

3.5.1 testTimestamp

- 测试项

```
android.media.audio.cts.AudioRecordTest#testTimestamp
```

- 报错

```
java.lang.AssertionError: AudioRecordTest expect maxAbsJitterMs(31.848597038199177)
&lt; 6.0
```

- 解决办法

```
//hardware/rockchip/audio/tinyalsa_hal
static struct pcm_config pcm_config_in = {
    #if PCM_REFERENCE_CHANNELS
    .channels = PCM_CAPTURE_CHANNELS + PCM_REFERENCE_CHANNELS,
#else
    .channels = PCM_CAPTURE_CHANNELS,
#endif
    .rate = 48000,
    .period_size = 480,   // 10ms
    .period_count = 4,
    .format = PCM_FORMAT_S16_LE,
};
```

.period_size改成240或者768

3.5.2 usb相关

- 测试项

```
android.media.audio.cts.AudioNativeTest#testRecordStreamData
android.media.audio.cts.AudioRecordSharedAudioTest#testCapturesMatch
```

- 报错

```
E modules.usbaudio.audio_hal: adev_release_audio_patch cannot find stream with patch
handle as 189
E modules.usbaudio.audio_hal: adev_create_audio_patch() the patch handle(202) does not
match recorded one(0) for stream with handle(5918) when creating audio patch
```

- 解决办法

hardware/libhardware/modules/usbaudio

```
diff --git a/modules/usbaudio/audio_hal.c b/modules/usbaudio/audio_hal.c
old mode 100644
new mode 100755
index fe921d69..43229938
--- a/modules/usbaudio/audio_hal.c
+++ b/modules/usbaudio/audio_hal.c
@@ -1338,6 +1338,7 @@ static int adev_open_input_stream(struct audio_hw_device
*hw_dev,
     int num_open_inputs = in->adev->inputs_open;
     device_unlock(in->adev);
+    num_open_inputs = 0;
     /* Check if an input stream is already open */
     if (num_open_inputs > 0) {
         if (!profile_is_cached_for(&device_info->profile, card, device)) {
@@ -1687,6 +1688,7 @@ static int adev_create_audio_patch(struct audio_hw_device *dev,
     if (are_devices_the_same(
                 num_configs, cards, devices, num_saved_devices, saved_cards,
saved_devices)) {
         // The new devices are the same as original ones. No need to update.
+        *patch_handle = *handle;
         stream_unlock(lock);
         return 0;
     }
```

3.5.3 hdmi相关

- 测试项

```
android.media.audio.cts.AudioManagerTest#testSetWiredDeviceConnectionState
```

- 分析

  机器接了hdmi屏但不支持声卡功能，sys/class/extcon下，对应的state为HDMI=1，会触发系统拔插检测，框架会添加hdmi音频设备，但实际硬件不支持

- 解决办法

  确认是否有如下修改，如果没有，请加上

```
diff --git a/services/core/java/com/android/server/WiredAccessoryManager.java
b/services/core/java/com/android/server/WiredAccessoryManager.java
index 46feee289e2f..854292a056e3 100644
--- a/services/core/java/com/android/server/WiredAccessoryManager.java
+++ b/services/core/java/com/android/server/WiredAccessoryManager.java
@@ -98,7 +98,12 @@ final class WiredAccessoryManager implements
WiredAccessoryCallbacks {
     private final InputManagerService mInputManager;
     private final boolean mUseDevInputEventForAudioJack;
-
+//-----------------------rk code----------
+    private boolean mHdmi0Registered = false;
```

```
+    private boolean mHdmi1Registered = false;
+    private boolean mDp0Registered = false;
+    private boolean mDp1Registered = false;
+//---------------------------------------
     public WiredAccessoryManager(Context context, InputManagerService inputManager) {
         PowerManager pm = (PowerManager)
context.getSystemService(Context.POWER_SERVICE);
         mWakeLock = pm.newWakeLock(PowerManager.PARTIAL_WAKE_LOCK,
"WiredAccessoryManager");
@@ -111,6 +116,28 @@ final class WiredAccessoryManager implements
WiredAccessoryCallbacks {
         mExtconObserver = new WiredAccessoryExtconObserver();
         mObserver = new WiredAccessoryObserver();
+//-----------------------rk code----------
+        String registered = mAudioManager.getParameters("hdmi0_registered");
+        if (registered.equals("hdmi0_registered=1")) {
+            Slog.d(TAG, "hdmi_0 registered");
+            mHdmi0Registered = true;
+        }
+        registered = mAudioManager.getParameters("hdmi1_registered");
+        if (registered.equals("hdmi1_registered=1")) {
+            Slog.d(TAG, "hdmi_1 registered");
+            mHdmi1Registered = true;
+        }
+        registered = mAudioManager.getParameters("dp0_registered");
+        if (registered.equals("dp0_registered=1")) {
+            Slog.d(TAG, "dp0 registered");
+            mDp0Registered = true;
+        }
+        registered = mAudioManager.getParameters("dp1_registered");
+        if (registered.equals("dp1_registered=1")) {
+            Slog.d(TAG, "dp1 registered");
+            mDp1Registered = true;
+        }
+//---------------------------------------
     }
     private void onSystemReady() {
@@ -320,19 +347,19 @@ final class WiredAccessoryManager implements
WiredAccessoryCallbacks {
                 outDevice = AudioManager.DEVICE_OUT_ANLG_DOCK_HEADSET;
             } else if (headset == BIT_USB_HEADSET_DGTL) {
                 outDevice = AudioManager.DEVICE_OUT_DGTL_DOCK_HEADSET;
-            } else if (headset == BIT_HDMI_AUDIO) {
+            } else if (headset == BIT_HDMI_AUDIO && mHdmi0Registered) {
 //-----------------------rk code----------
                 Slog.d(TAG, "hdmi_0 plug");
 //---------------------------------------
                 outDevice = AudioManager.DEVICE_OUT_HDMI;
 //-----------------------rk code----------
-            } else if (headset == BIT_HDMI_AUDIO_1) {
+            } else if (headset == BIT_HDMI_AUDIO_1 && mHdmi1Registered) {
                 Slog.d(TAG, "hdmi_1 plug");
                 outDevice = AudioManager.DEVICE_OUT_HDMI_1;
```

```
-            } else if (headset == BIT_DP_AUDIO) {
+            } else if (headset == BIT_DP_AUDIO && mDp0Registered) {
                 Slog.d(TAG, "dp_0 plug");
                 outDevice = AudioManager.DEVICE_OUT_SPDIF;
-            } else if (headset == BIT_DP_AUDIO_1) {
+            } else if (headset == BIT_DP_AUDIO_1 && mDp1Registered) {
                 Slog.d(TAG, "dp_1 plug");
                 outDevice = AudioManager.DEVICE_OUT_SPDIF_1;
```

3.6 CtsMediaV2TestCases

- 测试项目

```
android.mediav2.cts.AdaptivePlaybackTest#testAdaptivePlayback[0_c2.rk.av1.decoder_video/av01]
```

- 报错

```
E mpp_device: mpp_dev_init found unsupported client type 4 in platform 1009220a
E hal_av1d_api: mpp_dev_init failed ret: -6
```

- 解决办法

  dts添加av1d配置

```
av1d {
status = "okay";
};
```

3.7 MctsMediaBetterTogetherTestCases[instant]

- 测试项

```
android.media.bettertogether.cts.MediaSession2Test#testCallback_onPostConnect_rejected[INITIAL_USER]
```

- 解决办法

```
替换mainline版本至mainline_v_2025_jan_12974089.zip
```

3.8 MctsMediaV2TestCases

- 测试项

```
android.mediav2.cts.DecoderColorAspectsTest#testColorAspects[53_c2.android.vp9.decoder_video/x-vnd.on2.vp9_2_6_6]
```

- 报错分析

  测试case要求设备支持HDR

- 解决办法

  device/rockchip/common加上修改

```
diff --git a/modules/graphics.mk b/modules/graphics.mk
index 27e8e705..ff2bc8c7 100644
--- a/modules/graphics.mk
+++ b/modules/graphics.mk
@@ -25,7 +25,7 @@ endif
 #For CTS
 PRODUCT_PROPERTY_OVERRIDES += \
-    ro.surface_flinger.has_HDR_display=false
+    ro.surface_flinger.has_HDR_display=true
 PRODUCT_DEFAULT_PROPERTY_OVERRIDES += \
     ro.surface_flinger.game_default_frame_rate_override=60
```

3.9 CtsVideoCodecTestCases

- 测试项

```
android.videocodec.cts.VideoEncoderMultiResTest#testMultiRes[123_c2.rk.avc.encoder_video/avc_426x240_30fps_maxb-0_vbr_i-dist-1]
```

- 报错分析

  从编码输入buffer中获取到的stride是2对齐的，对于rk356x来说，rga输入要求stride至少4对齐

- 解决办法

  vendor/rockchip/hardware/interfaces/codec2

```
diff --git a/component/video/C2RKMpiEnc.cpp b/component/video/C2RKMpiEnc.cpp
index 5d4e328..162d58c 100644
--- a/component/video/C2RKMpiEnc.cpp
+++ b/component/video/C2RKMpiEnc.cpp
@@ -114,10 +114,23 @@ public:
         int64_t inputUsage = 0;
+        /*
+         * Some encoders have input alignment requirement, input buffer need
+         * rga conversion in first. detail see needRgaConvert(). so for fit rga
```

```
+         * compatibility, add following buffer usages:
+         * 1. allocate buffer within 4G to avoid rga2 error.
+         * 2. add the minimum RGA alignment in all platforms may needs rga
conversion.
+         */
          if (C2RKChipCapDef::get()->getChipType() == RK_CHIP_3588 ||
              C2RKChipCapDef::get()->getChipType() == RK_CHIP_356X) {
              inputUsage |= RK_GRALLOC_USAGE_WITHIN_4G;
          }
+         if (C2RKChipCapDef::get()->getChipType() != RK_CHIP_3588 &&
+             C2RKChipCapDef::get()->getChipType() != RK_CHIP_3562 &&
+             C2RKChipCapDef::get()->getChipType() != RK_CHIP_3576 &&
+             C2RKChipCapDef::get()->getChipType() != RK_CHIP_3528) {
+             inputUsage |= RK_GRALLOC_USAGE_STRIDE_ALIGN_16;
+         }
          addParameter(
                  DefineParam(mUsage, C2_PARAMKEY_INPUT_STREAM_USAGE)
```

4. GTS相关

4.1 GTS本地媒体包测试

- 使用本地的媒体包最大的好处就是能够排除网络问题，实现视频秒播。
- GTS测试中使用本地媒体包，可以访问Google说明源地址或从FTP服务器下载`GMS-Test-Suite/xTS/GTS_local_media_package`

1. 下载GtsExoPlayerTestCases、GtsYouTubeTestCases以及GtsMediaTestCases
2. push本地媒体包到机器sdcard

```
adb push gts test wvmedia /sdcard/
```

3. 使用命令测试：

```
run gts \
--module-arg GtsExoPlayerTestCases:config-url:https://storage.googleapis.com/exoplayer-test-media-1/gen-4/dynamic-config-sdcard-1.0.json \
--module-arg "GtsYouTubeTestCases:skip-media-download:true" \
--module-arg "GtsYouTubeTestCases:instrumentation-arg:media-path:=/sdcard/test" \
--module-arg "GtsMediaTestCases:instrumentation-arg:media-path:=file:///sdcard/wvmedia"
```

4. 模块单测命令(注意替换模块名`WvtsDeviceTestCases`)

```
run gts -m GtsExoPlayerTestCases \
--module-arg GtsExoPlayerTestCases:config-url:https://storage.googleapis.com/exoplayer-test-media-1/gen-4/dynamic-config-sdcard-1.0.json \
--module-arg "GtsExoPlayerTestCases:instrumentation-arg:media-path:=/sdcard/gts"

run gts -m GtsYouTubeTestCases \
--module-arg GtsYouTubeTestCases:config-url:https://storage.googleapis.com/exoplayer-test-media-1/gen-4/dynamic-config-sdcard-1.0.json \
--module-arg "GtsYouTubeTestCases:instrumentation-arg:media-path:=/sdcard/test"

run gts -m WvtsDeviceTestCases \
--module-arg WvtsDeviceTestCases:config-url:https://storage.googleapis.com/exoplayer-test-media-1/gen-4/dynamic-config-sdcard-1.0.json \
--module-arg "WvtsDeviceTestCases:instrumentation-arg:media-path:=file:///sdcard/wvmedia"
```

常见GTS测试fail项需要用本地媒体包测试

```
WvtsDeviceTestCases
com.google.android.wvts.WidevineH264PlaybackTests#testL3WithUHD60
com.google.android.wvts.WidevineH264PlaybackTests#testClearWithUHD60
com.google.android.wvts.WidevineVP9WebMPlaybackTests#testVP9WebMCencSuperFramesL3WithUHD30
com.google.android.wvts.WidevineVP9WebMPlaybackTests#testVP9WebMProfile210bitCencSuperFramesL3With1080P30
com.google.android.wvts.WidevineVP9WebMPlaybackTests#testVP9WebMCencSubSampleL3WithUHD60
```

4.2 常见fail项处理

4.2.1 GtsExoPlayerTestCases

- 测试项

  ```
  com.google.android.exoplayer.gts.DashStreamingTest#h264Adaptive
  ```

- 报错

  ```
   E TestRunner: Codec(DashStreamingTest:Video) was late decoding too many buffers.
  Counters:
   E TestRunner: DecoderCounters {
   E TestRunner:  decoderInits=1,
   E TestRunner:  decoderReleases=1
   E TestRunner:  queuedInputBuffers=3845
   E TestRunner:  skippedInputBuffers=0
   E TestRunner:  renderedOutputBuffers=3744
   E TestRunner:  skippedOutputBuffers=0
   E TestRunner:  droppedBuffers=101
   E TestRunner:  droppedInputBuffers=0
   E TestRunner:  maxConsecutiveDroppedBuffers=3
   E TestRunner:  droppedToKeyframeEvents=0
  ```

```
 E TestRunner:  totalVideoFrameProcessingOffsetUs=35288579
 E TestRunner:  videoFrameProcessingOffsetCount=3845
 E TestRunner: }:
 E TestRunner: expected to be at most: 39
 E TestRunner: but was               : 101
```

解码或者刷新慢，导致丢帧超过最大值

- 解决办法

```
device/rockchip/rk3576$ git diff package_performance.xml
diff --git a/package_performance.xml b/package_performance.xml
index 5b763ed..6b603af 100755
--- a/package_performance.xml
+++ b/package_performance.xml
@@ -3,4 +3,5 @@
 <app package="com.antutu.ABenchMark" mode="1"/>
 <app package="com.antutu.benchmark.full" mode="1"/>
 <app package="com.android.server.cts.device.graphicsstats" mode="1"/>
+<app package="com.google.android.exoplayer.gts" mode="1"/>
 </performance-package>
```

```
frameworks/native/services/surfaceflinger$ git diff
diff --git a/services/surfaceflinger/Scheduler/src/FrameTargeter.cpp
b/services/surfaceflinger/Scheduler/src/FrameTargeter.cpp
index b60aec49f6..2c27350d47 100644
--- a/services/surfaceflinger/Scheduler/src/FrameTargeter.cpp
++ b/services/surfaceflinger/Scheduler/src/FrameTargeter.cpp
@ -117,6 +117,8 @ void FrameTargeter::beginFrame(const BeginFrameArgs& args, const
IVsyncSource& v
mFramePending = false;
}
}
+ mFramePending = false;
```

4.2.2 WvtsDeviceTestCases

- 测试项

```
com.google.android.wvts.WidevineYouTubePerformanceTests#testClear1080P60UsingExo2Extra
ctor
```

- 报错

```
java.lang.Exception: Failed components: &lt; Performance Test: Device failed to meet
expectations. The mean weighted interframe time was 63.701025 where the expected value
was 16.666667. Some more useful information: mean=63.701025 variance=3554.755058
samples=1853 &gt;
```

- 解决办法

关闭副屏、保证主屏刷新率能到60

可通过如下命令，查看刷新率

```
setenforce 0 && setprop debug.sf.fps 1
logcat | grep mFps
```

滑动屏幕的时候，mFps的最大值必须要到60，如果到不了，比如最大只能到57，请提redmine，先解决屏幕刷新率问题，解决后再复测